Operators and Conditions

We have talked about operators in the past, specifically Arithmetic & Math operators. We are going to be covering some new ones today, and how to use these new ones as they may not be directly familiar like the math operators were. But before we get into that, let's do a quick reminder of all the different operators we have covered!

Arithmetic & Math Operators Review

What We Know

```
var foo = 10
var bar = 5

let sum = foo + bar
let difference = foo - bar
let product = foo * bar
let quotient = foo / bar
let remainder = foo % bar
```

Using the Math Operators In A New Way!

Okay, so we now have done a quick reminder of what the different operators are, and how we use them, now before getting into new ones, let's cover some other ways that we can use them.

Adding Strings

We can use the `+` operator for combing `Strings` together, for example:

Talk about String Interpolation as it was Skipped

```
let brotherOne = "Finn"
let brotherTwo = "Jake"

let brothers = brotherOne + " and " + brotherTwo + " are brothers."
```

Mixing Types

When you are using the `+` operator, it's important to note that by default, Swift doesn't support mixing types with addition / concatenation. So we can only add the same types together.

```
// ERROR: Cannot convert value of type `Int` to expected argument type `String`.
let cast: String = "There are " + cast.count + " Characters"
```

Adding Arrays

We have added `Strings` together before, but we can also add `Arrays` together using the same operator.

```
let boys = ["Harry", "Ron"]
let girls = ["Hermione", "Ginny"]

let cast = boys + girls
print(cast) // Harry, Ron, Hermione, Ginny
```

> ⚠ Warning
>
> This does **NOT** work for any of the other Collections we have covered.

Compound Operators

We also have our compound operators that we can use to shorten our math operations.

```
var score = 60

score += 12
score -= 3

score *= 2
score /= 3

// Score = 46

var quote = "Nothing is Impossible..."
quote += " Yet I do nothing every day."

print(quote) // Nothing is Impossible... Yet I do nothing every day.
```

Congrats! We made it through our review of the operators we currently know, now lets get into the new stuff!

Comparison Operators

Comparison operators are used to, get this, compare things. I'm sure you have seen them before, as we used them in math. If you need a refresher, try to remember the alligators eating the numbers. There are 3 basic comparison operators: `>`, `<`, `==`. When you use these comparison operators, they return a `Boolean`. So, like how the math operators return a number, these return a `Boolean`. They are comparing and telling you if the statement you are making is `true` or `false`.

```
let trueStatement = 1 < 2
let falseStatement = 1 > 2
```

Okay, so seeing if they are different is easy enough, now let's see if they are equal to each other.

```
// ERROR: Cannot assign to value: literals are not mutable
let checkEqual = 1 = 1
```

Uh-oh, that isn't going to work, we are getting an error that is saying: “Hey, you are trying to assign a value here, but I cannot do that”. We are getting this error because Swift uses the `=` as the assignment operator, so if we use `=` we are asking it to assign a value. So then, how would we see if two values are the same? We use a slightly different operator, instead of one `=` we use two like so `==`. This is the comparison operator for equal to.

```
let checkEqual = 1 == 1 // true
```

Compound Comparison Operators

Much like how our arithmetic operators had special compound operators, our comparison operators have that too, just in a slightly different manner. We can combine the less-than or greater-than operators with the equal to operator to get less-than / greater-than or equal to.

```
let lessThanEqualToTrue = 2 ≤ 2
let greaterThanEqualToFalse = 2 ≥ 3
```

Not

There is another big piece of our comparison operators. The bang `!` or Exclamation mark. We use this operator with a `Boolean` to get the opposite. So if you had `!true` the value would be `false`. Let's do an example:

```
let someBoolean = !(2 == 2) // false
```

Notice that I had to put parens around my `2 == 2` statement. Because we cannot say `not 2 equals 2` which is what `!2 == 2` would be. We have to say `not (2 == 2)` or in other words we need to solve the boolean first, then we can get the opposite. So the parens enforces the order of operations here.

For the above example, however, there is a simpler way. Checking if something doesn't equal something else is a super common operation, so it has its own operator `≠`. So the proper way to check the above would be to do this:

```
let notEqualExample = 2 ≠ 2
```

> **Note**
>
> This compound operator only works as `not equal` to use the bang with the other comparisons we would still need the bang like before

Comparing Non-Numeric Values

We can also compare much more than just numbers. Let's start by seeing if two `Strings` are the same value.

```
let stringOne = "StringOne"
let stringTwo = "StringTwo"
let areSameString = stringOne == stringTwo // false
```

All the types that we have covered so far can use the `==` equality operator like above, so long as they are the same type. When it comes to the other comparison operators, the story isn't quite the same. This comes down to the fact that most of the time it wouldn't be sensible to have a comparison other than `==` as it could be too broad as what that could mean. Without getting too deep into the specifics of the implementation. We would say that all the types we have covered are `Equatable` but not all are `Comparable`.

Conditional Statements

We have made `Booleans` out of numbers and letters, awesome, what purpose does that serve? Well, computers like to compare things, and we can choose to either do something or not do something based off of a `Boolean`. The simplest form of this would be called an `if` statement. When we use an `if` statement, we are saying `if some statement is true do this thing`.

The syntax for this kind of statement is:

1. `if` : The keyword we use to say we are creating an `if` statement.

2. Some `Boolean` value or expression that returns a `Boolean` (Comparison Operations).
3. Then we surround what code we want to run, if the statement is true, with curly braces.

> **Note**
>
> You are going to start seeing curly braces a lot, so get used to seeing and using them.

Let's do a super simple, hardly even `if` statement, so you can see the syntax first hand. We are going to write an `if` statement that will always run. We are also going to create an `if` statement that will never run.

```
if true {
    print("true")
}

if 2 == 1 {
    // WARNING: Will never be executed | Ignore This For Now As Thats The Point
    print("false")
}
```

If we run this code, you will see that our output is only true, no false. This is because since our second if statements `Boolean` value is equal to `false` so that means the block of code its using, won't get run.

Going back to the `bang` we can use that to make a block run when a statement isn't true.

```
if !(1 > 2) {
    print("true")
}
```

Technically speaking, you could also do a comparison between the `Boolean` values and get the same result, though you shouldn't ever do this in practice.

```
if (1 > 2) == false {
    print("true")
}
```

Else and Else If

If statements are incredibly powerful even on their own, and you will write a lot of code that uses them; however, they get better! There are two other elements that you can add to an `if` statement that extends its capabilities. `Else` and `Else If`. To show off the capabilities of these, let's write a bit of code.

```
// Blackjack
let firstCard = 11
let secondCard = 10

// Using Only Ifs
if firstCard + secondCard == 21 {
    print("Blackjack!")
}

if firstCard + secondCard == 2 {
    print("Aces")
}
```

In the above example, we know that if its value is `21` then it couldn't also be `2` right? So why would we run code that we already know the answer to? This is where `else if` can come in. `else if` comes after an `if` statement and only checks its condition if the prior `if` statement evaluates to `false`. So instead of what we have above, we would have this.

```
if firstCard + secondCard == 21 {
    print("Blackjack!")
} else if firstCard + secondCard == 2 { // Only Checks if the value
isn't 21
    print("Aces")
}
```

We can chain as many `else ifs` as we want, we could have a chain of 20 `else if` checks if we wanted.

> **Note**
>
> You should never have very many `else ifs`, we will cover the proper way to handle lots of comparisons at once.

There is also an `else` statement. `else` always comes last in one of these chains, this may be right after the first `if` or after several `else ifs`, but it will always be last. This is because `else` is a catch-all, meaning if none of the above statements was `true` then the `else` will run its code.

```
if firstCard + secondCard == 21 {
    print("Blackjack!)
} else if firstCard + secondCard == 2 {
    print("Aces")
} else {
    print("Normal Cards")
}
```

Adding the `else` statement gave us some extra functionality, but how would we do just the `else` statement without the `if` counterparts? Let's try and write some code that would do that, with just the knowledge we have now excluding `else if` and `else`.

We can write any code that we want within the `if` statement, so we could take an approach of doing what's called `nesting` where we write code inside those curly brace blocks that would check the value.

```
if firstCard + secondCard ≠ 21 {
    if firstCard + secondCard ≠ 2 {
        print("Normal Cards")
    }
}
```

AND / OR

This is a pretty messy solution, considering we only care about checking 2 values, this seems like an extreme approach. Luckily for a situation like this, we have a solution. When we are just checking conditions for multiple values, we can combine them into a single `Boolean` statement, using `&&` AND as well as `||` OR. So instead, we can do this.

```
if (firstCard + secondCard ≠ 2) && (firstCard + secondCard ≠ 21) {
    print("Normal Cards")
}
```

This combines the two statements into a single one, we are saying, if both of these statements are true, then run this code. We can combine as many of these `&&` statements together as we need, as long as all the values are `true` then we have a `true` statement. There is also the `||` statement. When we use the `||` statement we are saying that if either values on each side of the `||` are `true` then run the statement.

```
if 1 == 2 || 1 == 1 {
    print("true") // true
}
```

Since above, we are saying if `1 == 2 OR 1 == 1` it evaluates to `true` as `1 == 1` is `true`. You can combine these statements however you want, with all kinds of mixes of `ANDs` and `ORs`. I would highly recommend in these complex statements, to try to organize them using parens, or in other better ways we will go over later on.

Ternary Conditional Operations

Swift has a really cool way of shortening `if` / `else` statements in certain situations. When we do this, it's called a `Ternary Conditional Operation`. You will likely hear it called a `Ternary` for short. Let's start by writing a regular `if` statement that compares an answer to a question.

```
let answer = 42
let question = 42

if answer == question {
    print("Same")
} else {
    print("Different")
}
```

Now let's try to turn that statement into a `Ternary Operation`.

```
print(answer == question ? "Same" : "Different")
```

Spotless right? We turned a long 6 line statement into just a single line! A great way we can use this is when we are creating or updating variables and constants.

```
let xNum = 0
let yNum = 1

let biggerNum = xNum > yNum ? xNum : yNum
```

This syntax might be confusing for you, but as you read it more, you will get used to it. Just so I can really drill this into your heads, I'm going to break it down into pieces.

There are 3 main parts to a `Ternary Operation`: The Comparison, the `true` value, and the `false` value. It may help to see this breakdown by writing out something that would look more like an `if` statement, so let's write another `Ternary` example, and then write a comment that would be the `Ternary` as an `if`.

```
let xNumYNumBoolean = xNum > yNum ? true : false

```

```
// let xNumYNumBoolean = if xNum > yNum { true } else { false }
```

Now we wrote that out as a comment in the past; however, recently there was a new feature that was added to Swift, this feature being what we wrote in that comment. Starting in Swift 5.9 the ability to use an `if` statement as an `if` expression. Or in other words, let an if statement represent a value. This lets us use `if` / `else if` / `else` like a `Ternary` where we can assign a value using the `if` statement. One thing to note, is that when you do this, you always have to have an `else`.

```
let myRandomNumber = 3

let myLuckyStatement = if myRandomNumber == 1 {
    "You're lucky number one"
} else if myRandomNumber == 2 {
    "You are 2 lucky"
} else {
    "3, The highest and luckiest number of them all"
}
```

Switch Statements

Swift has yet another way to handle comparing things. `Switch` Statements. A `switch` is like a more powerful, faster, `if` / `else if` / `else` statement. However, it has to be exhaustive. Meaning, it has to handle every possible case.

```
let weather = "Sunny"

switch weather {
    case "Rainy":
        print("Bring an umbrella")
    case "Snowing":
        print("Grab your coat")
    case "Sunny":
        print("Wear your sunscreen")
    default: // We use default as an else, covering all non-covered
cases.
        print("What a wonderful day!")
}
```

Let's redo one of our previous `if` / `else if` / `else` example so you can see the differences better.

```
let cardOne = 11
let cardTwo = 10

switch (cardOne + cardTwo) {
     case 21:
          print("Blackjack")
     case 2:
          print("Aces")
     default:
          print("Nothing Special")
}
```

We can break down `switch` statements into a couple of major pieces.

```
switch <ValueToCompare> {
     case <PotentialValue>:
          <CodeToRun>
     default:
          <CodeToRun>
}
```

`Switch` statements give us a great way to compare many values all at once, instead of having super long `if` / `else if` / `else` chains. Let's do another example, we have a grade as a percentage, and we need to convert it to a letter grade.

```
let testScore = 90
switch testScore {
     case 90...: print("A")
     case 80...90: print("B")
     case 70...80: print("C")
     case 60...70: print("D")
     default: print("F")
}
```

There is some more new syntax here, we have the range syntax. When we put the `...` between two numbers, it's saying any number between these numbers. Or if we use only a single number, like in the case of `90...` that means either every number above or below this number based on the position of the dots. `90...` meaning 90 and above, or `...90` meaning up to and including 90.

You will also notice there is some overlap of these numbers, for example the number 80 appears in both the `B` and `C` statements, so how will swift use it? Well, just like our other

code, swift evaluates switch statements from top to bottom. So the number 80 would be caught by the `B` statement, meaning the `C` statement wouldn't get triggered.

Switch Statements and Enumerations

Alright, now I am sure you may be thinking you have seen some similar syntax. The syntax of `switch statements` is very similar to that of `enumerations`. This is important because of how well they work together! Most of the time if you are using an `enumeration` for your data, you will use a `switch` statement along with it!

Let's rewrite one of the `enumerations` that we wrote in the previous lesson, and then use a `switch statement` with it.

```
enum Activity {
     case bored
     case running
     case talking
     case coding
}

let currentActivity = Activity.coding

// Now lets switch on the value stored in our currentActivity constant.
switch currentActivity {
     case .coding: print("Hello, World!")
     case .talking: print("I can't stop talking")
     case .running: print("I am a healthy person")
     case .bored: print("I am bored")
}
```

Notice that unlike the other `switch statements`, I didn't have to put a `default` case. This is because we used all the possible cases, so there wasn't a need for a `default`. Since `enumerations` cover a set number of values, this is possible. You can, however, use a `default` case if you don't need / want to cover all the cases of an `enumeration`.

```
switch currentActivity {
     case .coding: print("I'm doing something fun!")
     default: print("I'm doing something boring...")
}
```

Associated Values With Switch Statements

Let's go back to when we were using `Associated Values`. I'm sure you remember, we didn't have any way to handle those values once we assigned them. Now with the

introduction of `switch` `statements` we can!

```
enum ValuedActivity {
    case bored
    case running(miles: Double)
    case talking(topic: String)
    case coding(lines: Int, hasBugs: Bool)
}

let runningActivity = ValuedActivity.running(miles: 5)
let codingActivity = ValuedActivity.coding(lines: 437, hasBugs: true)

switch runningActivity {
    case .bored:
        print("I'm Bored")
    case .running(let miles):
        print("I ran \(miles) miles.")
    case .talking: // Ignores the associated value
        print("I'm talking")
    case .coding(let lines, _): // ignores just the bugs value
        print("I coded \(lines) lines of code.")
}
```